```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cmath>
#include <iomanip>

namespace vole {

class Instruction {
private:
    unsigned short raw;
    int opcode;
    int first;
    int second;
    int third;
public:
    Instruction() : raw(0), opcode(0), first(0), second(0),
third(0) {}
    void set(int rawCode, int operation, int a, int b, int
c) {
        raw = static_cast<unsigned short>(rawCode);
        opcode = operation;
        first = a;
        second = b;
        third = c;
    }
    unsigned short getRaw() const { return raw; }
    int getOpcode() const { return opcode; }
    int getFirst() const { return first; }
    int getSecond() const { return second; }
    int getThird() const { return third; }
};

class InstructionDecoder {
private:
    int hexValue(char ch) const {
```

```
        if (ch >= '0' && ch <= '9') return ch - '0';
        if (ch >= 'A' && ch <= 'F') return ch - 'A' + 10;
        if (ch >= 'a' && ch <= 'f') return ch - 'a' + 10;
        return -1;
    }
public:
    bool decode(int rawCode, Instruction& instruction)
const {
        if (rawCode < 0 || rawCode > 0xFFFF) return false;
        int opcode = (rawCode >> 12) & 0xF;
        int first = (rawCode >> 8) & 0xF;
        int second = (rawCode >> 4) & 0xF;
        int third = rawCode & 0xF;
        if (opcode >= 1 && opcode <= 11) {
            if ((opcode == 4 || opcode == 10) && first !=
0) return false;
            instruction.set(rawCode, opcode, first, second,
third);
            return true;
        }
        if (rawCode == 0xC000) {
            instruction.set(rawCode, 12, 0, 0, 0);
            return true;
        }
        return false;
    }
    bool decodeHex(const std::string& text, Instruction&
instruction) const {
        if (text.length() != 4) return false;
        int value = 0;
        for (int i = 0; i < 4; i++) {
            int digit = hexValue(text[i]);
            if (digit < 0) return false;
            value = value * 16 + digit;
        }
        return decode(value, instruction);
    }
};
```

```
class Memory {
private:
    unsigned char cells[256];
public:
    Memory() {
        for (int i = 0; i < 256; i++) cells[i] = 0;
    }
    unsigned char read(int address) const {
        if (address < 0 || address >= 256) return 0;
        return cells[address];
    }
    void write(int address, unsigned char value) {
        if (address < 0 || address >= 256) return;
        cells[address] = value;
        if (address == 0xFF) std::cout <<
static_cast<int>(value) << std::endl;
    }
    bool loadBytes(int startAddress, const
std::vector<unsigned char>& bytes) {
        if (startAddress < 0 || startAddress +
static_cast<int>(bytes.size()) > 256) return false;
        for (int i = 0; i < static_cast<int>(bytes.size());
i++) cells[startAddress + i] = bytes[i];
        return true;
    }
    const unsigned char* getCells() const { return cells; }
};

class RegisterFile {
private:
    unsigned char registers[16];
public:
    RegisterFile() {
        for (int i = 0; i < 16; i++) registers[i] = 0;
    }
    unsigned char get(int index) const {
        if (index < 0 || index >= 16) return 0;
```

```
        return registers[index];
    }
    bool set(int index, unsigned char value) {
        if (index < 0 || index >= 16) return false;
        registers[index] = value;
        return true;
    }
    const unsigned char* getAll() const { return
registers; }
};

class ALU {
public:
    static unsigned char addTwoComplement(unsigned char a,
unsigned char b) {
        return static_cast<unsigned char>(a + b);
    }
    static unsigned char addFloatingPoint(unsigned char a,
unsigned char b) {
        int s1 = (a >> 7) & 1;
        int e1 = (a >> 4) & 7;
        int m1 = a & 15;
        int s2 = (b >> 7) & 1;
        int e2 = (b >> 4) & 7;
        int m2 = b & 15;
        double val1 = (m1 / 16.0) * std::pow(2.0, e1 - 4);
        if (s1) val1 = -val1;
        double val2 = (m2 / 16.0) * std::pow(2.0, e2 - 4);
        if (s2) val2 = -val2;
        double sum = val1 + val2;
        int resSign = 0;
        if (sum < 0) {
            resSign = 1;
            sum = -sum;
        }
        if (sum == 0) return 0;
        int exp = 4;
        while (sum >= 1.0 && exp < 7) {
```

```
            sum /= 2.0;
            exp++;
        }
        while (sum < 0.5 && exp > 0) {
            sum *= 2.0;
            exp--;
        }
        int mantissa = static_cast<int>(sum * 16.0 + 0.5);
        if (mantissa > 15) mantissa = 15;
        return static_cast<unsigned char>((resSign << 7) |
(exp << 4) | mantissa);
    }
    static unsigned char bitwiseOr(unsigned char a,
unsigned char b) { return a | b; }
    static unsigned char bitwiseAnd(unsigned char a,
unsigned char b) { return a & b; }
    static unsigned char bitwiseXor(unsigned char a,
unsigned char b) { return a ^ b; }
    static unsigned char rotateRight(unsigned char value,
int steps) {
        steps %= 8;
        if (steps == 0) return value;
        return static_cast<unsigned char>((value >> steps)
| (value << (8 - steps)));
    }
};

class Executor {
public:
    static void execute(const Instruction& instr,
RegisterFile& registers, Memory& memory, int& pc, bool&
halted) {
        if (halted) return;
        int op = instr.getOpcode();
        int r = instr.getFirst();
        int xy = (instr.getSecond() << 4) |
instr.getThird();
        int s = instr.getSecond();
```

```
        int t = instr.getThird();
        switch (op) {
            case 1: registers.set(r, memory.read(xy));
break;
            case 2: registers.set(r, static_cast<unsigned
char>(xy)); break;
            case 3: memory.write(xy, registers.get(r));
break;
            case 4: registers.set(t, registers.get(s));
break;
            case 5: registers.set(r,
ALU::addTwoComplement(registers.get(s), registers.get(t)));
break;
            case 6: registers.set(r,
ALU::addFloatingPoint(registers.get(s), registers.get(t)));
break;
            case 7: registers.set(r,
ALU::bitwiseOr(registers.get(s), registers.get(t))); break;
            case 8: registers.set(r,
ALU::bitwiseAnd(registers.get(s), registers.get(t)));
break;
            case 9: registers.set(r,
ALU::bitwiseXor(registers.get(s), registers.get(t)));
break;
            case 10: registers.set(r,
ALU::rotateRight(registers.get(r), t)); break;
            case 11:
                if (registers.get(r) == registers.get(0))
pc = xy;
                break;
            case 12: halted = true; break;
            default: break;
        }
    }
};

class Machine {
private:
```

```
    Memory memory;
    RegisterFile registers;
    InstructionDecoder decoder;
    int pc;
    bool halted;
public:
    Machine() : pc(0), halted(false) {}
    void reset() {
        memory = Memory();
        registers = RegisterFile();
        pc = 0;
        halted = false;
    }
    bool loadProgram(const std::vector<std::string>&
hexInstructions, int startAddress = 0) {
        std::vector<unsigned char> bytes;
        for (size_t i = 0; i < hexInstructions.size(); i++)
{
            Instruction instr;
            if (!decoder.decodeHex(hexInstructions[i],
instr)) return false;
            unsigned short raw = instr.getRaw();
            bytes.push_back(static_cast<unsigned char>(raw
>> 8));
            bytes.push_back(static_cast<unsigned char>(raw
& 0xFF));
        }
        if (!memory.loadBytes(startAddress, bytes)) return
false;
        pc = startAddress;
        return true;
    }
    void step() {
        if (halted) return;
        if (pc < 0 || pc >= 255) {
            halted = true;
            return;
        }
```

```
        unsigned char high = memory.read(pc);
        unsigned char low = memory.read(pc + 1);
        int rawCode = (high << 8) | low;
        pc += 2;
        Instruction instr;
        if (!decoder.decode(rawCode, instr)) return;
        Executor::execute(instr, registers, memory, pc,
halted);
    }
    void run() {
        while (!halted) step();
    }
};

}

int main() {
    vole::Machine machine;
    std::vector<std::string> program;
    program.push_back("2105");  // R1 = 5
    program.push_back("2203");  // R2 = 3
    program.push_back("5312");  // R3 = R1 + R2
    program.push_back("33FF");  // memory[FF] = R3  (بيطبع
8)
    program.push_back("C000");  // halt
    if (!machine.loadProgram(program)) {
        std::cout << "Invalid program" << std::endl;
        return 1;
    }
    machine.run();
    return 0;
}
```